David Tran
Sovannmonyrotn Kun
Oanh Tran
CECS 327-01

Project 2

Design

For this project we designed a “Campus Smart Parking Finder” that has:

- A concurrent parking server that maintains lot occupancy and reservations.
- A minimal RPC interface for mobile/terminal clients.
- An async update channel so sensors can push updates and the server can notify subscribers without blocking.
- A pub/sub mechanism so many clients can subscribe to lot updates without requiring polling

Architecture:

```
                    +-----------------------------------------------+
                    |                Parking Server                 |
                    |-----------------------------------------------|
                    |  In-Memory State (lots/reservations)|
                    |  Locks | Queues | Worker Threads   |
                    +-----------------------------------------------+
                       ^ RPC Connections     ^ Sensor Connections
                       |                     |
    +----------------+                       +-------------+
    |                                                     |
    v                                                     v
 +------------------------+                +------------------------+
 |     RPC Clients      |                 |     Sensors          |
 +------------------------+                +------------------------+
```

The server uses a thread-per-connection model for both RPC clients and sensor clients. This simplifies implementation and isolates blocking behavior: if one RPC client blocks while waiting for data, other connections remain unaffected.

Worker threads:

- RPC handler threads manage framed request/response flow
- Sensor handler threads enqueue updates (no shared blocking).
- A dedicated sensor update worker processes deltas sequentially, ensuring updates do not interfere with RPC handlers.

Synchronization:

- A global lock protects the shared parking lot state.
- Reservations store expiration timestamps; periodic checks ensure stale reservations are removed.

RPC:

- framing means packing length-prefixed messages to prevent sticking:
- Wire formatted in JSON
- Request: { "rpcId": 3, "method": "reserve", "args": [1, "ABC123"] }
- Reply: { "rpcId": 3, "result": true, "error": null }
- Timeouts: On timeout, it raises TimeoutError, completing the requirement for failure handling.
- interface uses a 4-byte big-endian length prefix

Asynchronous Messaging Path:

- Non-blocking sensor updates and client subscriptions
- Sensors connect on a **separate** TCP port and send: UPDATE <lotId> <delta>
- Ensures push notifications do not block regular RPC request/response flow
- The server:
    - Parses updates in their own handler threads
    - Pushes into a thread-safe queue
    - Applies in a single worker thread

Pub/Sub:

- pub/sub system so clients can subscribe to occupancy changes and receive notifications without polling
- subscribe(lotId) -> subId
- unsubscribe(subId) -> bool
    - subs/unsub a listener, user, or process to a specific lotId
- When a lot's free value changes it publishes: EVENT <lotId> <free> <timestamp>

Separate TCP connection for events:

- The client opens two connections:
    - RPC connection (request/reply)
    - Event connection (one-way async events)
- The server writes notifications to the event socket whenever events occur.
- RPC traffic stays clean and synchronous on its own socket.

Back Pressure:

- a policy for slow subscribers, if the server is saturated, applies queueing or reject with a clear error.
- Bounded per-subscriber queue + drop oldest
    - Each subscriber gets a queue with fixed capacity
    - If full, drop the oldest event to make room
    - Subscriber stays connected
    - System continues operating
    - Memory stays bounded

Evaluation

Sync RPC baseline + RPC tail latency and pub/sub event delivery behavior:

```
== Starting subscriber ==
== Starting sensor stress (10 updates/sec) ==

== Running RPC load tests (1/4/8/16 workers) ==

### Testing getAvailability
{'workers': 1, 'throughput': 100.9, 'median_ms': 13.21, 'p95_ms': 16.44}
{'workers': 4, 'throughput': 508.1, 'median_ms': 2.64, 'p95_ms': 24.82}
{'workers': 8, 'throughput': 1021.6, 'median_ms': 3.98, 'p95_ms': 24.8}
{'workers': 16, 'throughput': 285.5, 'median_ms': 55.21, 'p95_ms': 71.27}

### Testing reserve
{'workers': 1, 'throughput': 68.5, 'median_ms': 15.4, 'p95_ms': 28.54}
{'workers': 4, 'throughput': 195.1, 'median_ms': 18.93, 'p95_ms': 37.68}
{'workers': 8, 'throughput': 256.6, 'median_ms': 26.96, 'p95_ms': 65.75}
{'workers': 16, 'throughput': 291.0, 'median_ms': 54.49, 'p95_ms': 69.74}

== Tests complete. Subscriber output shows pub/sub behavior ==
Close with Ctrl+C.

== Tests complete. Shutting down subscriber and exiting ==
Done.
```

Async + pub/sub stress:

```
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487376538}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487376639}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487376739}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487376839}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487376939}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487377040}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487377140}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487377240}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487377341}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487377441}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487377542}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487377642}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487377743}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487377844}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487377944}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487378045}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487378145}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487378246}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487378346}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487378447}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487378547}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487378647}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487378748}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487378848}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487378948}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487379049}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487379149}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487379250}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487379351}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": 1, "occupiedPhysical": 1, "ts": 1772487379452}
INFO:parking:{"type": "sensor_update", "lotId": "A", "delta": -1, "occupiedPhysical": 0, "ts": 1772487379552}
```

Analysis:

Our synchronous RPC tests show that throughput scales with the number of parallel client workers, but not linearly. With 1-4 workers, the server maintains low median latency because each request is handled immediately by an available worker thread. At 8-16 workers, threads frequently block on socket reads/writes, and context-switching overhead increases. Blocking I/O combined with a fixed thread pool leads to queue buildup and elevated 95th-percentile latency under load.

When pub/sub sensors generate 10 events/sec per lot, the system experiences additional asynchronous load. Because event delivery is decoupled from RPC handling, synchronous request latency rises only slightly. The median stays stable, but the 95th percentile increases due to competition for CPU time and network bandwidth. This shows the trade-off between synchronous and asynchronous communication.

Fan-out and back-pressure policies strongly influence pub/sub behavior under stress. As subscriber queues fill, our drop-oldest policy prevents blocking on slow clients and ensures bounded memory use. During high loads, some event loss occurs but delivery of recent events remains timely for fast subscribers. The overall results confirm the messaging-system principle that decoupling improves scalability, but back-pressure prevents a single slow consumer from affecting global performance.

Reliability

Each sensor connection is monitored to where if no update or TCP activity is received within a certain amount of time, the system marks the sensor as disconnected. While disconnected, the server retains the last known availability value until either the sensor reconnects and resumes updates, or an operator or administrative RPC overrides the state. When the sensor eventually reconnects, it updates to the newest state.

Publish/Subscribe

Each update includes a timestamp, and the server applies only the latest valid update per lot. Older or duplicated updates are ignored. Reservation operations are made idempotent by associating each request with a unique client-generated operation ID, repeated RESERVE calls with the same ID return the same result without double-allocating spaces.